Project 14.1 — ReceiptRadar-Expense-Receipt-Extraction-And-Policy-Check

A practice twin of Project 14 — InvoiceIQ. Same skills — a document loader (Doc08), structured extraction (`with_structured_output`), and a **deterministic validator** — on a new real-world job. **There are no steps here on purpose.** Read the story, build it yourself, and open the hints or solution only if you get stuck.

Type: Document pipeline (load → extract → check policy → report) · **Stack:** Python, LangChain, `langchain-community` loaders, OpenAI SDK, Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

Every company that reimburses expenses has the same chore: someone reads each receipt, types in the merchant and amount, and checks it against the expense **policy** — was it over the limit that needs a written reason? was it a category the company won't pay for? It's slow, and the mistakes are either annoying (a valid receipt rejected) or expensive (a banned charge slips through).

You are going to build **ReceiptRadar**: a pipeline that reads a receipt, extracts what matters, and checks it against the policy before it's approved.

- A **loader** reads the receipt file (`.txt` / `.md` / `.pdf`) into text.
- An **extractor** turns it into a typed `Receipt` — merchant, amount, category, whether a note was attached — with `with_structured_output` .
- A **policy validator** runs the real rules: an amount over the note-threshold must have a note; banned categories (alcohol, gifts) are never reimbursable; required fields must be present. These are deterministic, so they're testable.
- A **reporter** returns the receipt plus any policy problems — empty means "approved."

The one lesson of this project: same as InvoiceIQ — the model *reads* the receipt, and your code *enforces the policy*. A policy is exactly the kind of rule you must never trust an LLM to apply consistently; write it as plain Python and it's right every time.

How it works, from a receipt to an approval (picture the whole machine)

A file path comes in. The **loader** reads it into text. The **extractor** sends that text to `model.with_structured_output(Receipt)`, so the reply is already a typed `Receipt`. The **policy validator** runs plain Python: compare the amount to the note-threshold, check the category against the banned list, confirm required fields. Each broken rule becomes a short problem string. The **reporter** bundles the `Receipt` and the problems into one result — empty list means approved.

That's the whole machine: **file → loader → extractor (typed Receipt) → policy validator → one result with any policy problems.**

What you are building (the pieces)

- A **loader** (`load_receipt_text(path) -> str`) using a Doc08 loader.
- A `Receipt` **Pydantic model** and `extract_receipt(model, text) -> Receipt`.
- A **policy validator** (`check_policy(receipt) -> list[str]`) with real rules.
- A **pipeline** (`process_receipt(model, path) -> Result`).

Inputs: a receipt file. **Outputs:** the receipt plus any policy problems (empty = approved). **Rules:** extract with structured output; enforce policy with deterministic code; an over-limit receipt with no note is always caught.

A real example to build and test against

- A small meal receipt ($12, with a note not required) → **approved**.
- A $200 hotel receipt with **no note** → one problem: "over the note threshold, needs a note".
- An "Alcohol" charge → one problem: "category not reimbursable".

How to practice this

1. Read this story. **Don't open the solution yet.** (Review Project 14 / Doc08 loaders / Doc05 structured output if hazy.)
2. Settle the **policy rules** on paper first — the note-threshold, the banned categories, the required fields. The policy is the point.
3. Build it: the `Receipt` model → the policy validator (test with hand-made receipts, no model) → the extractor → the loader → the pipeline.
4. Test it on the real example: an approved receipt, an over-limit-no-note one, a banned-category one.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

Policy enforcement over extracted data is everywhere — expenses here, but the same shape fits insurance claims, loan applications, content moderation, compliance checks. The model reads the messy input into a clean shape; your deterministic rules decide what's allowed. Build ReceiptRadar and you can put a trustworthy policy gate on top of any document-extraction system.

Done when

- A loader reads `.txt` / `.pdf` receipts into text.
- `extract_receipt` returns a typed `Receipt` via `with_structured_output`.
- `check_policy` catches an over-limit-no-note receipt and a banned category — proven with hand-made objects, no model.
- `process_receipt` returns the data plus any policy problems; a clean receipt is approved.
- Your tests pass: the policy validator (deterministic) and a live extraction.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.